

ЛАБОРАТОРНАЯ РАБОТА 3

3.1 Назначение и краткая характеристика работы

В процессе выполнения настоящей работы закрепляются знания студентов по машинным методам расчета переходных процессов курса «Вычислительные комплексы в электроэнергетике».

Целью работы является написание программы на Python, моделирующую переходные процессы.

3.2 Теоретическая справка

Git — распределённая система управления версиями.

При разработке с использованием git необходимо следовать следующим требованиям:

1. создание репозитория на git-сервере и клонирование его на локальный компьютер;
2. работа в отдельной ветке для каждой новой функциональности или исправления ошибок;
3. регулярный коммит изменений с понятными именами и описаниями;
4. использование pull request для объединения веток и проверки изменений другими участниками команды;
5. регулярное обновление локального репозитория из удаленного, чтобы избежать конфликтов при объединении веток.
6. использование тегов для пометки релизов и версий программного продукта;
7. резервное копирование репозитория на удаленном сервере или в облачном хранилище для защиты от потери данных.

3.3 Задание на работу в лаборатории

1. Разработать структуру конфигурационного файла. В нем должна содержаться полная информация о схеме и шаг расчета равный 50 мкс.
2. Реализовать парсер конфигурационного файла.
3. Реализовать решатель схемы.
4. Сформировать библиотеку компонентов.
5. Протестировать программу на последовательно- и паралельно-соединенных резисторах.

6. Протестировать программу на RL и RC схемах, и затем RLC схеме.

4. Методические указания к работе в лаборатории

4.1 Методические указания к п.2

Парсер возвращает матрицы A, E, J (при необходимости), Y и список всех объектов, описанных в схеме. Список всех объектов формируется в фабричном методе: на вход принимающий объект в формате JSON и возвращающий объект типа L, R, C.

4.2 Методические указания к п.3

Решатель отвечает за формирование системы уравнений, её решения и обновление внутренних параметров объектов (значение тока на элементе и потенциалов на концах элемента) и обновление системы уравнений. Решатель возвращает текущее состояние системы (матрицу потенциалов и матрицу токов).

Решатель схемы должен иметь метод `step()`, возвращающий две матрицы (матрицу потенциалов и матрицу токов). Таким образом пользователь программы сам определяет время моделирования схемы.

Решатель схемы обновляет состояние компонентов схемы путём итерирования по списку всех объектов, описанных в схеме. Аналогичным образом обновляется матрица E, которая используется для формирования системы уравнений.

При нахождении решений системы уравнений рекомендуется использовать алгоритм Доммеля:

1. Преобразовать исходную энергосистему в эквивалентную сеть, состоящую только из источников тока и резисторов.
2. Сформировать матрицу проводимостей для эквивалентной сети (рис. 10).
3. Используя данные предыдущего временного шага (или исходные условия для первого временного шага), рассчитать новые значения для токов [I].
4. Найти напряжения [V], используя новые значения тока [I].
5. Рассчитать токи в ответвлении, используя новые значения напряжения [V] и тока [I].
6. Повторить шаги 3-5.

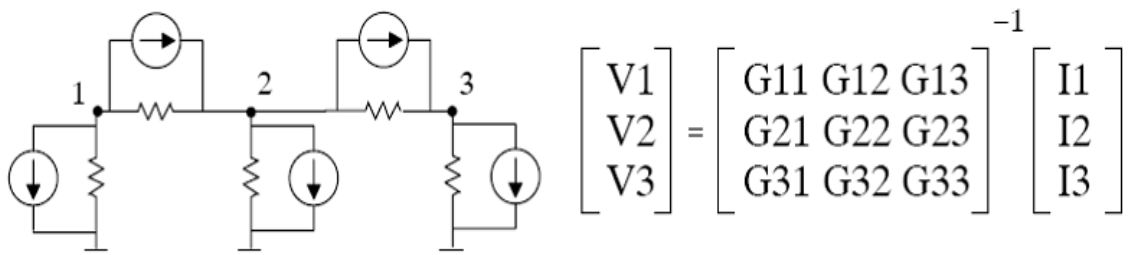


Рисунок 5 Иллюстрация по применению алгоритма Доммеля

4.3 Методические указания к п.4

Библиотека компонентов представляет собой отдельную директорию в которой разработчик создает классы, описывающие компоненты L, R, C. Причем все компоненты наследуются от абстрактного класса.

Абстрактный компонент должен содержать информацию о номерах узлах, к которым он подключен, номер ветки, к которой он принадлежит, абстрактные методы `get_E`, `get_R` и не абстрактные методы `set_fi_begin`, `set_fi_end`, `set_current`, `get_node_begin`, `get_node_end`.

5. Задание к составлению исполнительного отчета

Исполнительный отчет должен включать в себя:

1. титульный лист;
2. содержание конфигурационных файлов;
3. сравнение программно полученных графиков токов и напряжений и аналитически;
4. листинг кода;
5. выводы о проделанной работе.